

8교시: 디스크 연동 2 - named volume과 README evidence

수업 목표

- bind mount와 named volume의 차이를 설명한다.
- named volume에 쓴 데이터가 container 삭제 후에도 남는 흐름을 확인한다.
- 디스크 연동 실험 결과를 README에 build/run/disk/check/cleanup/troubleshoot 구조로 기록한다.

50분 흐름

시간	활동	비중	학생 산출
0-6분	bind mount vs named volume 비교	설명 12%	비교표
6-20분	named volume 생성과 write	실행 28%	write evidence
20-32분	새 container에서 read 확인	실행 24%	persistence evidence
32-40분	volume inspect/rm과 위험 설명	실행 16%	cleanup evidence
40-48분	README disk evidence 작성	실행 16%	README draft
48-50분	Day 3 연결	설명 4%	next note

Visual 1: named volume 데이터 보존

Week 2 Day 2
Lesson 8

Docker Named Volume 실험 & README 증거

컨테이너가 삭제되어도 데이터는 볼륨에 남아 유지됩니다.

1. Container A 생성 및 데이터 작성



/data (마운트)
/data/note.txt

```
$ docker run -d --name writer \
-v paperclip-day2-data:/data alpine sh -c "\
echo 'Hello from Day2!' > /data/note.txt && sleep 60"
```

/data/note.txt에 내용 저장
Hello from Day2!

2. Named Volume (Docker가 관리)



paperclip-day2-data

/
note.txt Hello from Day2!

3. Container A 삭제



삭제 후 유지
볼륨은 독립적으로 유지됩니다.

4. Container B 생성 및 데이터 읽기



/data (마운트)
/data/note.txt

```
$ docker run -it --rm --name reader \
-v paperclip-day2-data:/data alpine cat /data/note.txt
```

Hello from Day2!

같은 내용 확인
컨테이너가 달라도 동일한 데이터 사용 가능

Named Volume
데이터 보존

중요한 주의



volume rm은
데이터를 삭제합니다!

```
$ docker volume rm \
paperclip-day2-data
```

→ 데이터 영구 삭제!

민감정보 저장 금지



비밀번호, 토큰, 키 등
민감정보 저장 금지

RAW 바인드 마운트와 구분



Named Volume: Docker 관리
Bind Mount: 호스트 경로 직접 연결

실습 목표

- Named Volume 이해 및 사용
- 데이터가 컨테이너 삭제 후에도 보존됨을 확인
- 컨테이너 간 데이터 공유 확인
- 안전한 정리(Cleanup) 수행

다음 단계 (Day 3 준비)

- 네트워크 연결 (컨테이너 통신)
- 환경변수(ENV) 관리

핵심 정리

- Named Volume은 Docker가 관리하는 독립 스토리입니다.
- 컨테이너가 삭제되어도 데이터는 볼륨에 남아 유지됩니다.
- 여러 컨테이너가 동일 볼륨을 공유할 수 있습니다.
- volume rm을 실행하면 데이터가 영구 삭제됩니다!

권장 습관

- 사용 후 불필요한 볼륨은 정리하세요.
- 민감정보는 볼륨에 저장하지 마세요.

README 증거 (실행 기록 체크리스트)					
단계	명령 (Command)	결과 (Expected)	스크린샷/로그 파일	Volume 이름	정리 (Cleanup)
1 A: write	<pre>docker run -d --name writer \ -v paperclip-day2-data:/data alpine sh -c "\ echo 'Hello from Day2!' > /data/note.txt && sleep 60"</pre>	성공 (컨테이너 ID 생성)	_____	paperclip-day2-data	_____
2 A: 확인	<pre>docker exec writer cat /data/note.txt</pre>	Hello from Day2!	_____	paperclip-day2-data	_____
3 A: 삭제	<pre>docker rm -f writer</pre>	삭제 완료	_____	paperclip-day2-data	_____
4 B: read	<pre>docker run -it --rm --name reader \ -v paperclip-day2-data:/data alpine cat /data/note.txt</pre>	Hello from Day2!	_____	paperclip-day2-data	_____
5 볼륨 목록	<pre>docker volume ls</pre>	paperclip-day2-data 존재 확인	_____	paperclip-day2-data	_____
6 정리	<pre>docker volume rm paperclip-day2-data</pre>	삭제 완료	_____	(삭제됨)	완료 

이 이미지는 Container A가 named volume에 데이터를 쓰고 삭제된 뒤, Container B가 같은 volume에서 데이터를 읽는 흐름을 보여준다. container lifecycle과 data lifecycle이 분리된다는 점이 핵심이다.

bind mount와 named volume 비교

구분	Bind mount	Named volume
Source	host path를 직접 지정	Docker가 volume 이름으로 관리

적합한 용도	개발 중 host 파일 즉시 반영	DB/data처럼 container 삭제 후 보존
위험	host path 의존, OS별 path 차이	volume 삭제 전까지 데이터가 남음
삭제	host 파일은 직접 관리	docker volume rm으로 삭제

데이터 lifecycle 설명

container는 쉽게 만들고 지울 수 있어야 한다. 하지만 모든 데이터가 container와 함께 사라져도 되는 것은 아니다. database file, uploaded file, generated report처럼 보존이 필요한 데이터는 container lifecycle에서 분리해야 한다.

named volume은 Docker가 관리하는 persistent storage다. host path를 직접 외우지 않아도 volume name으로 mount할 수 있고, container가 삭제되어도 volume은 남는다. 이 특성은 DB 실습에서 중요하지만, 동시에 cleanup 책임도 만든다. "컨테이너를 지웠는데 왜 디스크가 계속 차지되는가"라는 질문은 대부분 volume이나 image/cache 정리와 연결된다.

학술/운영 관점

운영체제 관점에서 persistence는 "프로세스가 종료된 뒤에도 데이터가 남는가"의 문제다. container는 process lifecycle을 빠르게 만들고 삭제하는 데 적합하지만, storage는 별도의 lifecycle을 가진다. 그래서 운영 설계에서는 compute lifecycle과 data lifecycle을 분리해서 문서화한다.

lifecycle	Docker 예시	확인 명령
Process/container	docker run, docker stop, docker rm	docker ps, docker logs
Image artifact	docker build, docker images	docker history, docker inspect
Persistent data	docker volume create, mount	docker volume ls, docker volume inspect
Host file source	bind mount source path	pwd, ls, docker inspect

Hands-on 1: named volume 생성과 write

```
docker volume create paperclip-day2-data

docker run --rm \
  -v paperclip-day2-data:/data \
  alpine:3.20 \
  sh -c "echo volume-note-v1 > /data/note.txt && cat /data/note.txt"
```

Linux 사전 테스트 결과

```
paperclip-day2-data
volume-note-v1
```

첫 번째 줄은 volume 생성 결과이고, 두 번째 줄은 container가 volume 안에 쓴 파일을 읽은 결과다.

전체 named volume 실습은 [hands-on-lab.md](#)의 Phase G와 Phase H를 따른다. 이 교시에서는 volume을 만들고 읽는 것에서 끝내지 않고, inspect, cleanup, README evidence까지 완료한다.

Hands-on 2: 새 container에서 read 확인

```
docker run --rm \
  -v paperclip-day2-data:/data \
```

```
alpine:3.20 \
cat /data/note.txt
```

Linux 사전 테스트 결과

```
volume-note-v1
```

앞의 container는 `--rm` 으로 삭제되었지만 named volume은 남아 있었기 때문에 새 container가 같은 데이터를 읽을 수 있다.

Hands-on 3: inspect와 cleanup

```
docker volume inspect paperclip-day2-data
docker volume rm paperclip-day2-data
docker volume ls --filter name=paperclip-day2-data
```

주의: `docker volume rm` 은 volume 데이터를 삭제한다. DB 실습이나 실제 서비스 데이터에는 신중하게 사용해야 한다.

Linux 사전 테스트 cleanup 결과

```
docker stop paperclip-day2-disk
paperclip-day2-disk

docker rm paperclip-day2-disk
paperclip-day2-disk

docker volume rm paperclip-day2-data
paperclip-day2-data
```

정리 후 `docker ps --filter name=paperclip-day2-disk` 와 `docker volume ls --filter name=paperclip-day2-data` 는 헤더만 출력했다.

README disk evidence template

Disk Mount Evidence

Bind mount

- Source:
- Destination:
- Mode:
- v1 response:
- v2 response:
- Cleanup:

Named volume

- Volume name:
- Write command:
- Read command:
- Persistence result:
- Cleanup command:

Troubleshoot

Symptom	Evidence	Action
---	---	---

bind source path not found	pwd/ls/docker run error	source path 수정
response not updated	curl/browser cache/docker inspect	mount path와 browser cache 확인
volume data missing	docker volume ls/inspect	volume name 오타 또는 rm 여부 확인

추가 실습: volume inspect 읽기

```
docker volume inspect paperclip-day2-data
```

확인할 것:

- Name : paperclip-day2-data
- Driver : 보통 local
- Mountpoint : Docker가 관리하는 host-side 저장 위치

주의:

- Mountpoint 는 Docker 내부 관리 경로다. 수업에서는 이 경로를 직접 수정하지 않는다.
- 데이터 수정은 container를 통해 수행하고, 운영에서는 backup/restore 절차를 별도로 둔다.

평가 기준

기준	2점 evidence
비교	bind mount와 named volume 차이를 설명했다.
persistence	container 삭제 후 새 container가 volume 데이터를 읽는 것을 확인했다.
cleanup	volume 삭제 전 위험을 설명하고 정리했다.
README	disk evidence와 troubleshoot를 README에 기록했다.

공식 근거 링크

- Docker Docs: Volumes, <https://docs.docker.com/engine/storage/volumes/>
- Docker Docs: Bind mounts, <https://docs.docker.com/engine/storage/bind-mounts/>
- Docker Docs: Storage, <https://docs.docker.com/engine/storage/>

다음 연결

Day 3는 network, port, environment variable, volume을 더 넓게 다룬다. Day 2 후반의 디스크 연동 실습은 Day 3 volume/data persistence 수업의 선행 경험이다.